HIGH-LEVEL DESIGN (HLD)

Smart Track App

Architecture • Modules • APIs • Data Design • Security

Organized technical specification

Table of Contents

1. Introduction

2. System Design

3. API Catalogue

4. Data Design

5. Interfaces

6. Non-Functional Requirements

7. References

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document provides an architectural and system specification for Smart Track, a web-based retail management and transaction-tracking application designed for micro-retailers, such as mini-marts and sari-sari stores. It describes the high-level architecture, modules, data models, communication protocols, API specifications, data-management policies, and non-functional requirements. It serves as a technical blueprint for developers, system architects, and project stakeholders.

1.2 Intended Audience

· Software engineers and developers — implement backend controllers, database schemas, and RESTful API endpoints.

· UI/UX designers and frontend developers — align user-interface workflows with backend endpoints and payloads.

· System architects and lead developers — review architecture, component boundaries, and security/performance standards.

· Project evaluators and stakeholders — verify system scope, feature alignment, and functional compliance.

1.3 System Overview

Smart Track digitalizes daily store tracking and provides micro-retail owners with operational insights.

· Dashboard Analytics: displays today's net cashflow (income minus expenses), total inventory asset valuation, cash on hand across physical cash and digital e-wallets, and a seven-day transaction trend.

· Cashflow Management: records daily income and expense transactions categorized by payment channel, including Cash, GCash, and Bank Transfer.

· Inventory and Asset Valuation: catalogs products, tracks cost and selling prices, maintains stock levels, and calculates total active inventory asset value.

· Store Credit (Utang) Management: maintains customer debt balances, tracks repayment progress, and records settlements into cash accounts using transactional database operations.

2. System Design

2.1 Application Architecture

Smart Track uses a three-tier modular monolith architecture. This keeps system management straightforward while separating presentation, application logic, and data storage.

Layer	Responsibilities
Presentation Layer (Client)	Responsive web interface built with React/Figma components for fast data entry.
Application Layer (Server/Middleware)	API Gateway for routing, input validation, and session handling, with business-logic modules for Dashboard Analytics, Cashflow, Inventory, and Utang.
Data Layer (Database)	Relational database management system (MySQL or PostgreSQL) to maintain data integrity across financial ledgers.

2.2 Core Modules

· Dashboard Analytics Engine: Aggregates net cashflow, cash-on-hand balances, inventory asset value, and seven-day transaction trends.

· Cashflow Module: Creates and retrieves income and expense records and updates the appropriate cash account.

· Inventory Module: Maintains product catalog details, stock quantity, cost price, selling price, and product status.

· Utang / Store Credit Module: Tracks customer debt and processes payments while keeping credit, transaction, and cash-account records consistent.

· API Gateway / Middleware: Routes requests, verifies sessions, validates inputs, and applies authentication-related controls and CORS handling.

2.3 Key Process Flows

2.3.1 Daily Cashflow Logging

1. The user enters a new transaction, such as a sale or restocking expense.

2. The client sends the transaction payload to the API Gateway.

3. The Gateway validates the payload.

4. The Cashflow Module writes the entry to the transactions table and updates the corresponding cash_accounts balance.

5. The updated totals are returned to the client.

2.3.2 Store Credit (Utang) Settlement

6. The customer pays toward an outstanding balance.

7. The client sends POST /api/v1/credits/pay.

8. The API Gateway verifies the session.

9. The Utang Module starts an ACID transaction.

10. The system reduces the balance in store_credits, adds a record to transactions, and updates cash_accounts.

11. The database commits the changes and returns HTTP 200 OK with the updated balance.

2.3.3 Inventory Asset Value Update

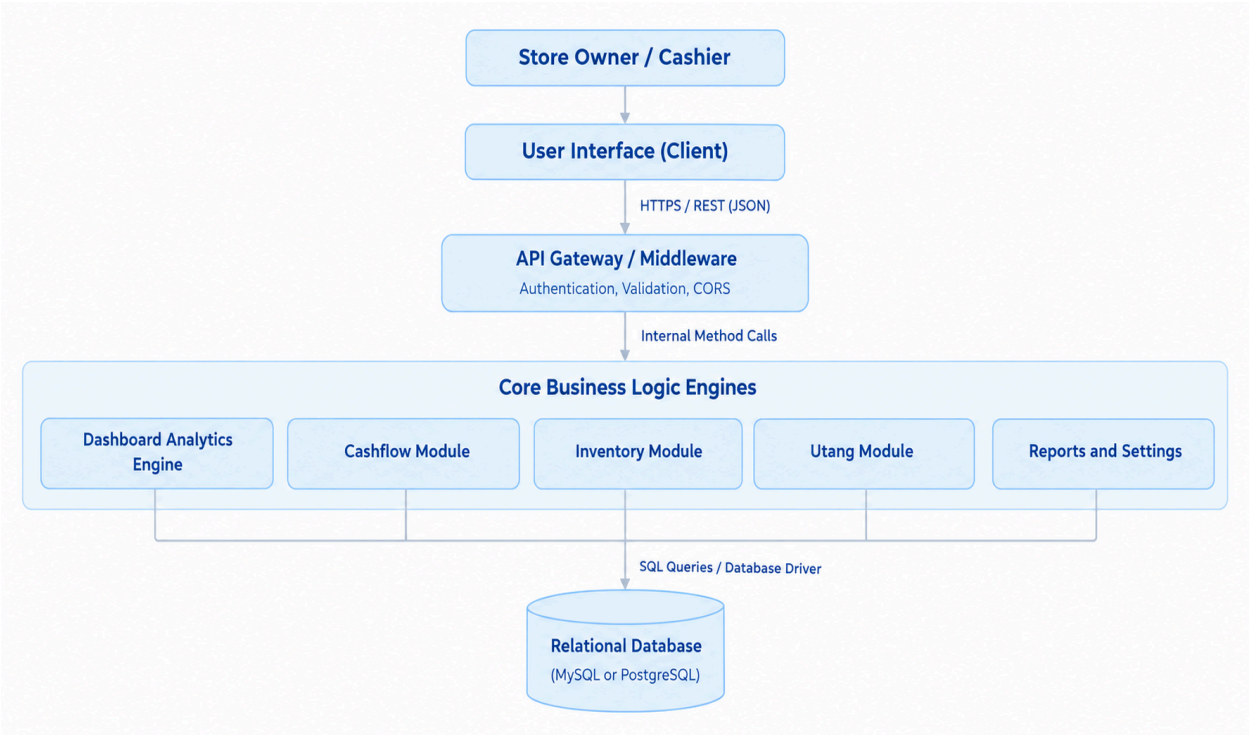
12. A product's stock quantity or unit cost price changes.

13. The Inventory Module updates the products table.

14. The Dashboard Analytics Engine calculates total inventory value using cost_price × stock_quantity across active products.

15. The dashboard KPI is refreshed.

2.4 Information Flow



3. API Catalogue

The API exposes REST endpoints over HTTPS and uses JSON request/response payloads.

Operation	Method	Endpoint	Parameters / Payload	Purpose
Get Dashboard Summary	GET	/api/v1/dashboard/summary	store_id (query)	Returns net cashflow, cash-on-hand split, and total asset value.
Get Daily Chart Data	GET	/api/v1/cashflow/daily-chart	store_id, days	Returns grouped income and expense data for a bar chart (seven days by default/expected use).
List Transactions	GET	/api/v1/transactions	store_id, page, limit	Retrieves a paginated list of recent transactions.

Create Transaction	POST	/api/v1/transactions	{store_id, type, amount, category, payment_method, description}	Logs an income or expense transaction.
List Inventory Items	GET	/api/v1/inventory/products	store_id	Returns the product catalog with stock counts and prices.
Update Product Stock	PUT	/api/v1/inventory/products/{id}	{stock_quantity, cost_price, selling_price}	Updates stock quantity and cost/selling prices.
Process Credit Payment	POST	/api/v1/credits/pay	{credit_id, amount, payment_method}	Settles customer debt using an atomic multi-table transaction.

4. Data Design

4.1 Core Data Model

The relational data model contains five core entity tables.

Table	Main Fields	Purpose
stores	id (PK), name, location	Parent entity for store-owned records.
products	id (PK), store_id (FK), name, cost_price, selling_price, stock_quantity, status	Stores product details and stock.
cash_accounts	id (PK), store_id (FK), type, balance	Tracks balances by account type, such as physical cash or e-wallet.
transactions	id (PK), store_id (FK), type, amount, category, payment_method, transaction_date	Records income and expense activity.
store_credits	id (PK), store_id (FK), customer_name, amount_due, status	Tracks customer credit balances and settlement status.

4.2 Relationships

· One store can have many products.

· One store can have many cash accounts.

· One store can have many transactions.

· One store can have many store-credit records.

4.3 Data Access and Integrity

· ORM / Query Builder: use parameterized queries or ORM/database abstraction drivers to reduce SQL injection risk.

· Transaction Management: multi-table operations, including Utang payments, run inside explicit ACID transaction blocks (BEGIN … COMMIT) and roll back automatically on failure.

· Indexing: index key foreign keys such as store_id and search fields such as transaction_date and status to support queries as data grows.

4.4 Data Retention

· Active transaction ledger: retain cashflow transactions and credit histories online for at least three years for year-over-year reporting.

· Archival: compress and move transaction logs older than three years to cold storage or a cold database backup.

· Soft deletes: use flags such as is_deleted = TRUE for product or store deletion to preserve financial audit history.

4.5 Data Migration and Initial Seeding

· Schema version control: maintain sequential, version-controlled migration scripts (for example, V1__init_schema.sql and V2__add_payment_methods.sql).

· Initial seeding: automatically create default store settings and cash-account structures (physical_cash and e_wallet) when an account is activated.

5. Interfaces

Interface	Description
User Interface	Responsive single-page web application optimized for desktop, tablet, and mobile. Main views: Dashboard, Cashflow (In/Out), Inventory & Assets, Store Credit (Utang), Reports, and Settings.
API Gateway Interface	RESTful HTTPS endpoints accepting and returning JSON (application/json).
Database Interface	Standard TCP connection using secured credentials over a local socket or private VPC connection.

6. Non-Functional Requirements

6.1 Security

· Encryption in transit: encrypt client-server communication using TLS 1.3 / HTTPS.

· Encryption at rest: encrypt database storage volumes using AES-256.

· Authentication and authorization: require JWTs or secure session cookies in request headers for API access.

· Input sanitization: validate and sanitize incoming API parameters at the API Gateway to help prevent SQL injection (SQLi) and cross-site scripting (XSS).

6.2 Performance and Scalability

· Response latency: dashboard summary endpoint (/api/v1/dashboard/summary) should respond in less than 200 ms under standard operating conditions.

· Database throughput: target up to 100 concurrent read/write requests per second per store instance.

· Scalability: stateless API server instances can be horizontally scaled behind a load balancer as demand grows.

7. References

16. Smart Track Figma Wireframes & UI Prototypes (October 2026).

17. RESTful API Design Standards & HTTP/1.1 Specifications (RFC 7231).

18. Database ACID Compliance & Relational Database Design Principles.